

Terminal und erste Befehle

Shell, Prompt und Pfade verstehen. Ordner und Dateien anlegen, kopieren, umbenennen, löschen und ansehen. Mit Platzhaltern viele Dateien auf einmal fassen.

Warum das wichtig ist

Auf einem Server gibt es keine Maus. Du meldest dich an und siehst nur Text. Alles, was du dort tust, tust du mit Befehlen.

Auch auf dem Desktop bist du mit dem Terminal schneller. Drei Ordner anlegen dauert mit der Maus 15 Sekunden. Mit einem Befehl dauert es fünf Sekunden.

Heute lernst du das Terminal kennen und die wichtigsten Befehle für Dateien und Ordner.

Terminal, Shell und Prompt

Öffne das Terminal mit **Strg+Alt+T**. Du siehst ein Fenster mit Text und einem blinkenden Cursor. Dieses Fenster ist das **Terminal**. Es nimmt an, was du tippst, und zeigt Text auf dem Bildschirm. Mehr macht es nicht. Die Befehle selbst versteht es nicht.

Tippe `ls` und drücke Enter. Es erscheint eine Liste deiner Ordner. Wer hat geantwortet? Die **Shell**. Sie ist das Programm, das im Terminal läuft. Sie liest deinen Befehl, führt ihn aus und schickt das Ergebnis an das Terminal zurück. Unsere Shell heißt **Bash**.

Merke dir den Unterschied: Das Terminal ist das Fenster. Die Shell ist das Programm dahinter. Im Alltag sagen viele Leute Terminal und meinen die Shell, oder umgekehrt. Es sind aber zwei Dinge.

	Terminal	Shell
Aufgabe	Nimmt deine Tastatureingabe an und zeigt Text an. Versteht die Befehle nicht.	Liest den Befehl, führt ihn aus, schickt das Ergebnis zurück.
Kümmert sich um	Schrift, Farben, Fenstergröße, Tabs	Was bei <code>cd</code> oder <code>ls</code> passiert

	Terminal	Shell
Beispiele	QTerminal (bei Lubuntu), Windows Terminal	Bash (Linux), PowerShell und cmd (Windows)

Warum zwei Dinge? Der Grund liegt in der Geschichte.

Früher war das Terminal ein Gerät. Ein Bildschirm mit Tastatur, sonst nichts. Es konnte selbst keine Programme ausführen. Es hing per Kabel an einem großen Rechner im Nebenraum. So sah ein Terminal aus: das DEC VT100 von 1978.



DEC VT100, ein Terminal von 1978



IBM System/360, ein Mainframe aus den 1960ern

Der große Rechner hieß Mainframe. Auf ihm lief die Shell. Er war so groß wie ein Zimmer und sehr teuer. Viele Terminals hingen gleichzeitig an einem Mainframe. Jeder Benutzer sah an seinem Terminal nur den Text, den die Shell schickte.

Heute ist das Terminal ein Programm, das so ein Gerät nachahmt. Darum heißt es genau genommen Terminal-Emulator. Die Trennung ist geblieben: Das Fenster ist das eine, das Programm dahinter das andere. Unter Windows kennst du das vielleicht. Im Fenster Windows Terminal läuft mal PowerShell, mal cmd. Das Fenster bleibt gleich, nur die Shell wechselt.

So läuft es immer ab: Du tippst einen Befehl und drückst Enter. Das Terminal gibt ihn an die Shell. Die Shell führt ihn aus und schickt das Ergebnis zurück. Das Terminal zeigt es an. Dann wartet die Shell auf den nächsten Befehl.

Schau dir jetzt die Zeile am Anfang genauer an:

```
carlos@lubuntu:~$
```

Das ist der **Prompt**, also die Eingabeaufforderung. Er sagt dir vier Dinge:

- `carlos` ist dein Benutzername.
- `lubuntu` ist der Name des Rechners.
- `~` ist der Ordner, in dem du gerade bist. Die Tilde `~` steht für dein Home-Verzeichnis.
- `$` heißt: du bist ein normaler Benutzer. Bei `#` wärst du root, also Administrator.

Ein Befehl hat immer denselben Aufbau:

```
befehl -optionen argumente
```

Zum Beispiel `ls -l /etc`. `ls` ist der Befehl, `-l` ist die Option, `/etc` ist das Argument. Ein Argument ist das, womit der Befehl arbeitet: eine Datei, ein Ordner oder ein Text. Beides ist nicht Pflicht. `pwd` kommt ohne Option und ohne Argument aus. `echo "Hallo Linux"` hat keine Option, aber ein Argument: den Text.

Wo bin ich, was ist hier?

```
pwd
# /home/carlos

ls
# Desktop  Dokumente  Downloads  linux-kurs  Musik  Bilder

ls -l
# insgesamt 24
# drwxr-xr-x 2 carlos carlos 4096 Sep 21 09:12 Desktop
# -rw-r--r-- 1 carlos carlos 253 Sep 21 09:14 notizen.txt

ls -a
# .  ..  .bashrc  .profile  Desktop  Dokumente  notizen.txt
```

`ls -l` zeigt Details: Rechte, Besitzer, Größe, Datum. `ls -a` zeigt auch versteckte Dateien. Versteckte Dateien fangen mit einem Punkt an, zum Beispiel `.bashrc`. Optionen kannst du kombinieren: `ls -la`.

Pfade

Ein Pfad ist die Adresse einer Datei.

Ein **absoluter Pfad** beginnt mit `/` . Er gilt immer, egal wo du gerade bist. Beispiel:

`/home/carlos/notizen.txt` .

Ein **relativer Pfad** beginnt nicht mit `/` . Er zählt ab deinem aktuellen Ordner. Beispiel:

`dokumente/notizen.txt` .

Drei Abkürzungen brauchst du ständig:

Zeichen	Bedeutung	Beispiel
<code>.</code>	der aktuelle Ordner	<code>cp datei.txt .</code>
<code>..</code>	der Ordner eine Ebene höher	<code>cd ..</code>
<code>~</code>	dein Home-Verzeichnis	<code>cd ~</code>

```
cd /etc
```

```
pwd
```

```
# /etc
```

```
cd ..
```

```
pwd
```

```
# /
```

```
cd ~
```

```
pwd
```

```
# /home/carlos
```

```
cd -
```

```
# /
```

`cd -` springt zurück zum vorherigen Ordner. `cd` ohne alles bringt dich direkt nach Hause.

Die wichtigsten Befehle

Befehl	Was er tut	Beispiel
<code>pwd</code>	zeigt den aktuellen Ordner	<code>pwd</code>
<code>ls</code>	listet den Inhalt auf	<code>ls -la /etc</code>

Befehl	Was er tut	Beispiel
<code>ls -R</code>	listet auch alle Unterordner auf	<code>ls -R projekte</code>
<code>cd</code>	wechselt den Ordner	<code>cd /var/log</code>
<code>mkdir</code>	legt einen Ordner an	<code>mkdir projekte</code>
<code>mkdir -p</code>	legt auch fehlende Oberordner an	<code>mkdir -p a/b/c</code>
<code>touch</code>	legt eine leere Datei an	<code>touch notizen.txt</code>
<code>cp</code>	kopiert	<code>cp notizen.txt kopie.txt</code>
<code>cp -r</code>	kopiert einen ganzen Ordner	<code>cp -r projekte backup</code>
<code>mv</code>	verschiebt oder benennt um	<code>mv alt.txt neu.txt</code>
<code>rm</code>	löscht eine Datei	<code>rm kopie.txt</code>
<code>rm -r</code>	löscht einen Ordner mit Inhalt	<code>rm -r backup</code>
<code>cat</code>	zeigt eine ganze Datei	<code>cat notizen.txt</code>
<code>head</code>	zeigt die ersten Zeilen	<code>head -n 5 notizen.txt</code>
<code>tail</code>	zeigt die letzten Zeilen	<code>tail -n 5 notizen.txt</code>
<code>echo</code>	gibt Text aus	<code>echo "Hallo Linux"</code>
<code>clear</code>	leert den Bildschirm	<code>clear</code>
<code>history</code>	zeigt deine letzten Befehle	<code>history</code>
<code>man</code>	zeigt die Anleitung zu einem Befehl	<code>man ls</code>

Ein kurzes Beispiel am Stück:

```
mkdir projekte
cd projekte
touch liste.txt
echo "Hallo Linux"
# Hallo Linux
```

```
cp liste.txt liste2.txt
mv liste2.txt kopie.txt
ls
# kopie.txt  liste.txt
rm kopie.txt
ls
# liste.txt
```

Achtung: `rm` löscht sofort und endgültig. Es gibt keinen Papierkorb. Prüfe den Namen zweimal, bevor du Enter drückst.

Dateien ansehen

Oft willst du nur kurz sehen, was in einer Datei steht. Du musst sie dafür nicht öffnen. Drei Befehle reichen.

`cat` zeigt die ganze Datei auf einmal. Gut für kurze Dateien.

```
cat /etc/os-release
# PRETTY_NAME="Ubuntu 22.04.4 LTS"
# NAME="Ubuntu"
# VERSION_ID="22.04"
# ...
```

`head` zeigt nur den Anfang, `tail` nur das Ende. Ohne Option sind es zehn Zeilen. Mit `-n` wählst du die Anzahl.

```
head -n 5 ~/.bashrc
# die ersten fünf Zeilen der Datei

tail -n 5 ~/.bashrc
# die letzten fünf Zeilen der Datei
```

`cat` steht für concatenate, also aneinanderhängen. Gibst du zwei Dateien an, klebt `cat` sie zusammen:

```
cat teil1.txt teil2.txt .
```

Tipp: Eine Datei ist sehr lang und `cat` rauscht durch? Dafür gibt es `less`. Damit blätterst du in

Ruhe. Das lernst du an Tag 3.

Platzhalter

Ein Platzhalter steht für einen oder mehrere Buchstaben. Damit fasst du viele Dateien auf einmal an. Die Shell setzt die passenden Namen ein, noch bevor der Befehl läuft.

Zeichen	Bedeutung	Beispiel
*	beliebig viele Zeichen, auch keins	*.txt passt auf alle .txt-Dateien
?	genau ein Zeichen	bild?.png passt auf bild1.png , nicht auf bild12.png
[...]	ein Zeichen aus der Liste	datei[12].txt passt auf datei1.txt und datei2.txt

Ein Beispiel. Erst der Inhalt, dann die Platzhalter:

```
ls
# bild1.png bild2.png liste.txt notiz.txt test1.log test2.log

ls *.txt
# liste.txt notiz.txt

ls bild?.png
# bild1.png bild2.png

rm test*.log
ls
# bild1.png bild2.png liste.txt notiz.txt
```

So kopierst du alle Textdateien mit einem einzigen Befehl:

```
mkdir sicherung
cp *.txt sicherung/
```

Achtung: Sei vorsichtig mit `rm` und Platzhaltern. `rm *` löscht alles im Ordner. Prüfe erst mit `ls *muster*` , was der Platzhalter trifft. Passt es, tausche `ls` gegen `rm` .

Namen richtig schreiben

Zwei Dinge bringen am Anfang fast alle durcheinander.

Groß und klein ist nicht dasselbe. `Notiz.txt` und `notiz.txt` sind zwei verschiedene Dateien. Linux achtet genau auf große und kleine Buchstaben. Windows tut das nicht.

Leerzeichen im Namen sind heikel. Die Shell trennt am Leerzeichen. `mkdir Mein Ordner` legt zwei Ordner an: `Mein` und `Ordner`. Willst du einen Ordner mit Leerzeichen im Namen, gibt es zwei Wege:

```
mkdir "Mein Ordner"
mkdir Mein\ Ordner
```

Die Anführungszeichen sind der leichtere Weg. Der Backslash `\` schützt genau das eine Zeichen dahinter.

Tipp: Nimm im Kurs am besten gar keine Leerzeichen in Namen. Ein Unterstrich `_` oder ein Bindestrich `-` ist einfacher. Also `mein_ordner` statt `mein ordner`.

Hilfe holen

Du musst nichts auswendig lernen. Zwei Wege führen zur Anleitung:

```
ls --help
# Usage: ls [OPTION]... [FILE]...

man ls
```

`man` öffnet das Handbuch. Du blätterst mit den Pfeiltasten, suchst mit `/suchwort` und beendest mit `q`.

Tipp: Du kennst den Namen des Befehls nicht mehr? `man -k kopieren` sucht im Handbuch nach einem Stichwort.

Tricks, die Zeit sparen

- **Tab** vervollständigt Datei- und Ordernamen. Tippe `cd Dok` und drücke Tab. Daraus wird `cd Dokumente/`.

- **Pfeil hoch** holt den letzten Befehl zurück. Nochmal drücken holt den davor.
- **Strg+C** bricht das laufende Programm ab.
- **Strg+L** leert den Bildschirm, genau wie `clear`.
- **Strg+D** beendet das Terminal, genau wie `exit`.
- **Strg+R** durchsucht deine alten Befehle. Tippe ein Stück des gesuchten Befehls. Enter führt ihn aus, Strg+C bricht die Suche ab.
- **Kopieren und Einfügen** geht im Terminal mit **Strg+Umschalt+C** und **Strg+Umschalt+V**. Das normale Strg+C bricht ja ab.
- `history` zeigt alle Befehle der Sitzung mit Nummer.

Tip: Tab ist der wichtigste Trick von allen. Er spart Tipparbeit und verhindert Tippfehler in Dateinamen.

Häufige Fehler

Diese Fehler machen am Anfang fast alle. Kennst du sie, sparst du dir Ärger.

- **rm hat keinen Papierkorb.** Was du löschst, ist weg. Bei `rm -r` und `rm *` zweimal hinsehen.
- **Groß und klein verwechselt.** `cd Dokumente` ist nicht `cd dokumente`. Tab hilft, den richtigen Namen zu treffen.
- **cp ohne -r bei einem Ordner.** Das geht nie, auch nicht bei einem leeren Ordner. Linux sagt dann: `cp: -r not specified; omitting directory`. Für einen Ordner brauchst du immer `cp -r`.
- **Leerzeichen im Namen vergessen.** `mkdir Mein Ordner` gibt zwei Ordner. Nimm Anführungszeichen oder einen Unterstrich.
- **Verlaufen im Dateibaum.** Unsicher, wo du bist? `pwd` sagt es dir. `ls` zeigt, was hier liegt.

Mach mit

So haben wir es im Unterricht gemacht. Tippe alles selbst ab, nicht kopieren.

1. Öffne das Terminal mit **Strg+Alt+T**.
2. Tippe `pwd`. Du bist in `/home/<dein name>`.
3. Tippe `ls` und danach `ls -la`. Vergleiche die beiden Ausgaben.
4. Sieh dir eine Datei an: `cat /etc/os-release`. Da steht, welche Distribution du nutzt.
5. Zeige Anfang und Ende deiner Shell-Einstellungen: `head -n 5 ~/.bashrc` und `tail -n 5 ~/.bashrc`.

6. Lege einen Ordner an: `mkdir uebung` .
7. Wechsle hinein: `cd uebung` . Prüfe mit `pwd` .
8. Lege drei Dateien an: `touch a.txt b.txt c.txt` . Prüfe mit `ls` .
9. Gib Text aus: `echo "Hallo Linux"` . Der Text erscheint direkt im Terminal.
10. Kopiere: `cp a.txt kopie.txt` . Benenne um: `mv kopie.txt neu.txt` . Prüfe mit `ls` .
11. Probiere einen Platzhalter: `ls *.txt` . Du siehst nur die Textdateien.
12. Lege einen Ordner mit Leerzeichen an: `mkdir "Mein Ordner"` . Prüfe mit `ls` .
13. Lösche: `rm neu.txt` . Prüfe mit `ls` .
14. Gehe eine Ebene hoch: `cd ..` . Prüfe mit `pwd` .
15. Wechsle mit absolutem Pfad: `cd /etc` . Liste mit `ls` auf.
16. Gehe nach Hause: `cd ~` oder nur `cd` .
17. Probiere Tab: Tippe `cd ue` und drücke Tab.
18. Drücke Pfeil hoch, bis du `mkdir uebung` wieder siehst.
19. Suche mit **Strg+R** nach `touch` . Du findest deinen Befehl von vorhin.
20. Tippe `history` . Du siehst alles, was du heute getippt hast.
21. Leere den Bildschirm mit `clear` oder **Strg+L**.

Bildnachweise

- DEC VT100: Foto von Jason Scott, Lizenz CC BY 2.0. Quelle: Wikimedia Commons.
- IBM System/360: Foto von Erik Pitti, Lizenz CC BY 2.0. Quelle: Wikimedia Commons.

Aufgaben

Arbeitsordner anlegen

Lege dir einen festen Platz für den Kurs an.

1. Gehe in dein Home-Verzeichnis.
2. Lege den Ordner `linux-kurs` an, falls er noch nicht da ist.
3. Lege darin den Unterordner `tag02` an. Nutze dafür einen einzigen Befehl mit der Option `-p`.
4. Wechsle in `tag02`.
5. Lege die drei leeren Dateien `a.txt`, `b.txt` und `c.txt` an. Nutze dafür einen einzigen Befehl.
6. Prüfe das Ergebnis mit `ls -l`.

✓ Bestätigt

Kopieren, umbenennen, löschen

Arbeite im Ordner `~/linux-kurs/tag02`.

1. Kopiere `a.txt` zu `a-kopie.txt`.
2. Benenne `a-kopie.txt` in `notizen.txt` um.
3. Lege den Ordner `alt` an und verschiebe `b.txt` hinein.
4. Lösche `c.txt`.
5. Kopiere den ganzen Ordner `alt` nach `alt-backup`.
6. Zeige zum Schluss alles mit `ls -R` an.

✓ Bestätigt

Hilfe selbst finden

Du sollst zwei Optionen von `ls` selbst herausfinden.

1. Öffne die Kurzhilfe mit `ls --help`.
2. Suche die Option, die nach Änderungsdatum sortiert.
3. Suche die Option, die Dateigrößen gut lesbar anzeigt, also zum Beispiel `4,0K` statt `4096`.
4. Öffne man `ls` und suche darin mit `/` nach `sort`.
5. Führe zum Schluss einen Befehl aus, der beide Optionen zusammen mit `-l` benutzt.

✓ Bestätigt

Absolut und relativ navigieren

Bewege dich durch das Dateisystem und prüfe nach jedem Schritt mit `pwd`, wo du bist.

1. Wechsle mit einem absoluten Pfad nach `/var/log`.
2. Gehe mit einem relativen Pfad eine Ebene höher.
3. Wechsle mit einem relativen Pfad wieder nach `log`.
4. Springe mit einer Abkürzung in dein Home-Verzeichnis.
5. Wechsle mit einem relativen Pfad nach `linux-kurs/tag02`.
6. Springe mit `cd -` zurück in den Ordner davor.

✓ Bestätigt

Arbeitsordner von Hand bauen

Baue diese Struktur in deinem Home-Verzeichnis. Nur mit Befehlen, nicht mit der Maus.

```
mein_arbeitsordner/  
  dokumente/  
    notizen.txt  
  skripte/  
    test.sh
```

1. Lege `mein_arbeitsordner` an und wechsele hinein.
2. Lege die beiden Unterordner `dokumente` und `skripte` mit einem einzigen Befehl an.
3. Lege in `dokumente` die leere Datei `notizen.txt` an.
4. Lege in `skripte` die Datei `test.sh` an.
5. Prüfe alles mit `ls -R`.
6. Kopiere zum Schluss den ganzen Ordner `mein_arbeitsordner` nach `backup_arbeitsordner`.

✓ Bestätigt

Dateien ansehen

Sieh dir Dateien an, ohne sie zu öffnen.

1. Zeige die ganze Datei `/etc/os-release`. Da steht, welche Distribution du nutzt.
2. Zeige nur die ersten fünf Zeilen von `/etc/os-release` mit `head`.
3. Zeige nur die letzten drei Zeilen von `~/.bashrc` mit `tail`.
4. Finde selbst heraus: Wie viele Zeilen zeigt `head`, wenn du keine Option angibst? Probiere `head ~/.bashrc`.

✓ Bestätigt

Platzhalter üben

Übe die Platzhalter in einem neuen Ordner.

1. Lege `~/linux-kurs/tag02-glob` an und wechsele hinein.
2. Lege mit einem einzigen `touch` diese Dateien an: `bild1.png`, `bild2.png`, `bild3.png`, `notiz.txt`, `liste.txt`.
3. Zeige nur die `.png`-Dateien mit einem Platzhalter.
4. Zeige nur die `.txt`-Dateien mit einem Platzhalter.
5. Lege den Ordner `bilder` an und kopiere alle `.png`-Dateien mit einem Befehl hinein.
6. Prüfe erst mit `ls bild?.png`, welche Dateien das Muster trifft. Lösche sie dann mit `rm bild?.png`.

✓ Bestätigt

Abschluss-Challenge

Zeig, dass alles von heute sitzt. Nur mit Befehlen, nicht mit der Maus.

1. Baue mit einem einzigen `mkdir -p` die Ordner `~/linux-kurs/projekt/texte` und `~/linux-kurs/projekt/bilder`.
2. Wechsle nach `~/linux-kurs/projekt/texte`.
3. Lege mit einem `touch` die Dateien `kapitel1.txt`, `kapitel2.txt`, `kapitel3.txt` und `titel.md` an.
4. Kopiere alle `.txt`-Dateien in den Ordner `bilder` eine Ebene höher. Nutze einen Platzhalter und einen relativen Pfad.
5. Wechsle nach `~/linux-kurs/projekt` und zeige alles mit `ls -R`.
6. Lege dort einen Ordner mit Leerzeichen im Namen an: `Alte Dateien`.
7. Verschiebe `titel.md` aus `texte` in diesen Ordner. Denk an die Anführungszeichen.

✓ Bestätigt

Quiz

Das Quiz gibt keine Punkte. Du kannst es so oft wiederholen, wie du willst.

Quiz abgegeben. Ergebnis: 9 / 9 richtig**1. Was zeigt der Befehl `pwd` an?**

Das Passwort des Benutzers

Den Ordner, in dem du gerade bist (Deine Antwort)

Alle Dateien im aktuellen Ordner

Den Namen des Rechners

Richtig

Erklärung:

`pwd` heißt print working directory. Es gibt den vollständigen Pfad des aktuellen Ordners aus.

2. Was macht `cd ..` ?

Es wechselt ins Home-Verzeichnis

Es wechselt eine Ebene nach oben (Deine Antwort)

Es wechselt ins Wurzelverzeichnis /

Es bleibt im gleichen Ordner

Richtig

Erklärung:

`..` steht für den übergeordneten Ordner. `.` steht für den aktuellen Ordner.

3. Wofür steht das Zeichen ~ in einem Pfad?

Für das Wurzelverzeichnis /

Für dein Home-Verzeichnis, also /home/benutzername (Deine Antwort)

Für den aktuellen Ordner

Für einen Platzhalter, der auf alle Dateien passt

Richtig

Erklärung:

~ ist die Abkürzung für dein Home-Verzeichnis. `cd ~` und `cd` bringen dich beide dorthin.

4. Welche Taste vervollständigt Datei- und Ordernamen?

Enter

Tab (Deine Antwort)

Strg

Esc

Richtig

Erklärung:

Tab vervollständigt. Zweimal Tab zeigt alle passenden Namen an.

5. Was macht Strg+C im Terminal?

Es kopiert den markierten Text

Es bricht das laufende Programm ab (Deine Antwort)

Es leert den Bildschirm

Es schließt das Terminal

Richtig

Erklärung:

Strg+C bricht ab. Kopieren im Terminal geht mit Strg+Umschalt+C.

6. Welcher Befehl zeigt auch die versteckten Dateien an?

Deine Antwort: `ls -a`

Richtig

Erklärung:

`ls -a` zeigt alles, auch Dateien, die mit einem Punkt beginnen. Mit `-l` kommen die Details dazu.

7. Welcher Befehl erstellt die leere Datei `notizen.txt` ?

Deine Antwort: `touch notizen.txt`

Richtig

Erklärung:

`touch` legt eine leere Datei an. Gibt es die Datei schon, ändert `touch` nur das Datum.

8. Welcher Befehl löscht den Ordner `backup` samt Inhalt?

Deine Antwort: `rm -r backup`

Richtig

Erklärung:

`rm -r` löscht einen Ordner mit allem darin. Es gibt keinen Papierkorb, der Inhalt ist weg.

9. Du bist in einem Ordner mit den Dateien `a.txt`, `b.txt` und `c.log`. Was zeigt `ls *.txt`?

Alle drei Dateien

Nur `a.txt` und `b.txt` (Deine Antwort)

Nur `c.log`

Eine Fehlermeldung

Richtig

Erklärung:

Der Platzhalter `*` steht für beliebig viele Zeichen. `*.txt` trifft alle Namen, die auf `.txt` enden.

Bonusaufgaben

Freiwillig, für alle, die früher fertig sind. Bonusaufgaben zählen nicht für die Tagesmedaille. Sind alle Bonusaufgaben eines Tages bestätigt, gibt es eine Bonusmedaille.

Platzhalter-Rätsel Bonus

Übe `*`, `?` und `[]` mit vielen Dateien.

1. Lege `~/linux-kurs/tag02-raetse1` an und wechsele hinein.
2. Lege mit einem einzigen `touch` diese 13 Dateien an: `log1.txt` bis `log12.txt` und `alt.log`.
3. Zeige nur die Dateien mit einer einstelligen Zahl, also `log1.txt` bis `log9.txt`. Tipp: `?` steht für genau ein Zeichen.
4. Zeige nur die Dateien mit einer zweistelligen Zahl.
5. Zeige nur `log1.txt`, `log3.txt` und `log5.txt` mit einem einzigen Muster. Tipp: `[135]`.
6. Lege den Ordner `zehner` an und verschiebe alle zweistelligen Dateien mit einem Befehl hinein.
7. Lösche `log2.txt`, `log4.txt`, `log6.txt` und `log8.txt` mit einem einzigen `rm` und `[]`.
8. Zeige zum Schluss alles mit `ls -R`.

Sicherung und Wiederherstellung Bonus

Ein Admin sichert, bevor er löscht. Spiel das einmal durch.

1. Kopiere den ganzen Ordner `~/linux-kurs/tag02` nach `~/linux-kurs/tag02-sicherung`.
2. Vergleiche beide Ordner mit `ls -R`. Sie müssen gleich aussehen.
3. Lösche das Original `~/linux-kurs/tag02` samt Inhalt.
4. Prüfe mit `ls ~/linux-kurs`, dass es weg ist.
5. Stelle es aus der Sicherung wieder her. Der Ordner muss danach wieder `tag02` heißen.
6. Prüfe mit `ls -R`, dass alles zurück ist.
7. Lösche die Sicherung.
8. Überlege: Warum ist `cp -r` für eine Sicherung besser als `mv`?

man-Detektiv Bonus

Finde vier Optionen selbst heraus. Nutze `man` oder `--help`. Schreibe zu jeder Option einen Satz auf Papier.

1. Was macht `ls -S` ?
2. Was macht `ls -r` ? Achtung: kleines `r` , nicht `-R` .
3. Was macht `cp -v` ?
4. Was macht `rm -i` ?
5. Probiere alle vier in `~/linux-kurs/tag02` aus: `ls -lS` , `ls -lSr` , `cp -v a.txt a2.txt` und `rm -i a2.txt` .